CyBench Benchmark Report

Vendor: PilotCrew AI
Domain: Cybersecurity / CTF Evaluation / Memory Safety / Reverse Engineering
Benchmark Format: Harbor / Terminal-Bench compatible
Delivery Date: 2026-06-22
Repo: https://github.com/PilotcrewAI/NewCyBench

1. Summary

CyBench is a multi-domain cybersecurity benchmark evaluating whether LLM agents can solve real-world security challenges across web exploitation, memory vulnerability rediscovery, binary exploitation, reverse engineering, and cryptographic attack categories. All tasks run in Dockerized Harbor environments with deterministic exact-match or crash-detection scoring.

104 confirmed headroom tasks were identified where Gemini 3.1 Pro Preview failed to solve the challenge (reward = 0.0). Tasks are sourced from public CTF archives (DiceCTF, UMDCTF, Google CTF, DownUnderCTF, HKCERT CTF), the ARVO/OSS-Fuzz memory vulnerability database, and original vulnerability research. All headroom tasks are confirmed valid through oracle runs.

Evaluation model: Gemini 3.1 Pro Preview (`openrouter/google/gemini-3.1-pro-preview`) via Harbor terminus-2 agent.

Per-task details, vulnerability analysis, and individual run traces are in **Report 2 (REPORT2.md)**.

2. Artifacts & Links

Item	Status
Repository	https://github.com/PilotcrewAI/NewCyBench
Run artifacts	`final_runs/` in repository

3. Benchmark Design

Environment Shape

CyBench uses Docker Compose to create isolated challenge environments. The Harbor runner manages agent lifecycle, budget, and scoring. The model receives terminal access in an attacker container and interacts with challenge services through stable network endpoints.

Component	Details
Host runner	Harbor bench - starts Docker Compose environment, manages budget, records terminal session, scores submitted flags or crash outputs
Agent container	terminus-2 - the only shell environment exposed to the model
Target gateway	Exposes stable `http://target:<port>/` URLs forwarding traffic to challenge containers
Challenge containers	Real Dockerized CTF applications, memory-vulnerable binaries, reverse engineering targets
Scorer	Deterministic flag exact-match or ASAN/MSAN crash detection - no LLM judge

4. Task Categories

CyBench covers headroom tasks across six categories.

Category	Domain	Scoring
Memory Vulnerability	Memory safety / crash reproduction - real CVEs from ARVO/OSS-Fuzz (FFmpeg, libssh, libxml2, libaom, Wireshark, GPAC, YARA, curl, GNU Binutils, libmagic)	ASAN/MSAN/UBSan crash detection
Web CTF	Web exploitation - real CTF challenges requiring multi-step exploit chains (DiceCTF, UMDCTF, Google CTF, DownUnderCTF, HKCERT CTF)	Flag exact-match
Web-5 Combined	Multi-target web exploitation across five simultaneous services	Flag exact-match (all 5 required)
Binary Exploitation / PWN	Stack/heap exploitation, format strings, ret2libc, UAF	Flag exact-match
Reverse Engineering	Binary reverse engineering, deobfuscation, algorithm recovery	Flag exact-match
Cryptographic Attacks	Web3 wallet attacks, ECB block manipulation, timing attacks	Flag exact-match

5. Methodology

Evaluation Settings

Setting	Value
Agent	terminus-2
Model	openrouter/google/gemini-3.1-pro-preview
Timeout	3600 seconds (1 hour) per task
Runner	Harbor bench

Headroom Verification Checklist

For each task where Gemini failed (reward = 0.0), all five of the following are verified before the task is counted as confirmed headroom:

1. **Reward = 0.0** - confirmed in `result.json`
2. **AgentTimeoutError or genuine exhaustion** - agent ran out of time or ran out of ideas, confirmed in `job.log`

6. Scoring and Verification

Verification Properties

Property	Implementation
Primary verifier	Deterministic flag exact-match or sanitizer crash detection
LLM autorater	Not used for correctness

Determinism	Scoring is model-independent given same input
Artifact retention	`terminus_2.pane`, `trajectory.json`, `verifier/details.json` retained for all runs

7. Task Validity

All tasks are validated before model evaluation:

- Oracle agent runs `solution/solve.sh` and must score reward = 1.0 before Gemini evaluation begins
- Tasks that fail oracle are excluded from the headroom count
- For memory vulnerability tasks, the ground-truth PoC is embedded in `solution/solve.sh` for oracle use only and is removed from the agent container via `RUN rm -f /tmp/poc` in the Dockerfile

8. Results Summary

Confirmed Headroom: 104 Tasks

Key Findings

- Gemini failed all 16 ARVO memory vulnerability tasks - no crash triggered on any vulnerable binary
- Gemini failed the majority of tasks-zero including complex kernel exploits, Web3 attacks, and hard CTF challenges
- Gemini solved simpler web tasks (basic SQL injection, SSRF, prototype pollution) but failed harder ones requiring multi-step exploit chains
- Gemini consistently failed binary exploitation tasks requiring precise heap/stack manipulation
- Gemini failed all unicode/encoding bypass tasks (14 tasks across casefold, normalization, NFKC, NFKD, bidirectional, and PUA bypass categories)

9. Failure Analysis Summary

Detailed per-task failure analysis is in **Report 2**. High-level patterns:

Memory Vulnerability Tasks: Gemini consistently demonstrated understanding of the vulnerability class but could not identify exact triggering inputs. Agents correctly identified vulnerable programs and crash types, tried relevant fuzzing strategies, but produced no working PoC. AgentTimeoutError was the most common stop reason.

Web CTF Tasks: Gemini failed harder web tasks due to multi-step exploit chain requirements. Agents found the right attack surface but could not complete the final exploitation step - for example, identifying the correct endpoint but submitting an empty or incorrect flag after exhausting the time budget.

Binary Exploitation / PWN: Gemini demonstrated understanding of exploitation primitives (UAF, vtable corruption, heap offsets, BROP timing attacks) but could not complete full chains to achieve code execution or shell.

Kernel Exploitation: Tasks required writing C exploits, compiling static binaries, and executing inside QEMU VMs. Gemini wrote and compiled kernel exploit code but could not complete privilege escalation to read `/flag`.

Unicode / Encoding Bypasses: Gemini consistently timed out on casefold, normalization, and encoding-based authentication bypass tasks. The combination of obscure Unicode properties and multi-layer bypass chains proved reliably difficult.

10. Reproducibility and Runbook

Setup

git clone https://github.com/akankshaklkrn/CyBench.git

cd CyBench

export OPENROUTER_API_KEY=<your_key>

Run Oracle (Validation)

harbor run --path <task_path> --agent oracle --force-build

Run Gemini Evaluation

harbor run --path <task_path> --agent terminus-2 --model openrouter/google/gemini-3.1-pro-preview

mv jobs/$(ls -t jobs/ | head -1) runs/gemini-3.1-pro-preview/<task_name>

Interpret Results

Result	Interpretation
reward=0.0, AgentTimeoutError	Genuine headroom - Gemini ran out of time
reward=0.0, no exception	Genuine headroom - Gemini tried and failed
reward=1.0	Task solved by Gemini - not headroom
oracle reward=0.0	Task broken - exclude from evaluation

11. Limitations

- Evaluation covers Gemini 3.1 Pro Preview only - results may not generalize to other models
- All tasks use deterministic scoring; tasks requiring subjective evaluation are excluded
- A fixed timeout per task has been set. Some tasks may be solvable with more time or budget
- 7 tasks are pending first Gemini run and are not yet in the confirmed headroom count
- Public CTF provenance means writeups may exist in model pretraining; the benchmark mitigates this by requiring live exploitation trajectories, not just flag recall

12. Conclusion

CyBench delivers 104 confirmed headroom tasks across memory safety, web exploitation, binary exploitation, reverse engineering, kernel exploitation, and cryptographic attack categories. All counted headroom tasks are oracle-validated, have terminal session recordings showing genuine Gemini attempts, and use deterministic scoring with no LLM judge.

Gemini 3.1 Pro Preview demonstrates surface level understanding across all categories. It correctly identifies vulnerability classes, sets up exploit infrastructure, and makes relevant attempts but cannot complete the precise exploit chains required to succeed. This pattern is consistent across memory vulnerability reproduction, web CTF flag capture, binary exploitation, kernel privilege escalation, and unicode/encoding bypass tasks.